

SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 (BL3, BL6)	Type:	Hands-on / Lab-embedded
Assessment:	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)		
Weightage:	7 marks of 100 (Internal / CA)	Total Marks:	1 × 100 = 100 (1 Question)
Schedule:	20-Aug-2026 (Day 16)	Duration:	Within 2-hr lab
CO3 Statement:	Build Retrieval-Augmented Generation (RAG) pipelines and AI agents by integrating LLM APIs, embeddings, and vector databases to develop intelligent real-world applications (BL3, BL6)		
Student Name:	S.DHARANI	Reg. No.:	192472364
Section / Batch:	2024-2028	Date:	13-08-26

Instructions to Students

- Answer **1 question only** — the question whose number matches your allotted Register Number order (1–40), as announced by the faculty. Total: 100 Marks.
- Use the API platform specified in your question (Hugging Face / OpenAI / Gemini). If access is unavailable, inform the faculty before starting; do not switch platforms on your own.
- Set up authentication (API key / token) securely — keys must be loaded from environment variables or a config file, never hard-coded in the submitted code.
- Implement the complete request → response flow: prepare the input, call the API with appropriate parameters, parse the response, and present the output clearly.
- Include error handling for at least: invalid/missing input, API failure or timeout, and rate-limit / quota errors.
- Demonstrate the working program with the test inputs specified in your question, including the edge case mentioned.
- Comment your code and record the model name, endpoint, and key parameters (temperature, max tokens, etc.) you used.
- Briefly justify your choice of model and parameters for the given problem.

Submission Requirements

- Source code / Jupyter Notebook
- Execution screenshots showing inputs and outputs (including the edge case)
- Record of model, endpoint, and parameters used
- Brief README / documentation with setup steps and design justification

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

EXPERIMENT 30 — TRAVEL ITINERARY DRAFTER USING GEMINI API

1. Aim

To develop a **Travel Itinerary Drafter** using the **Google Gemini API** that generates a structured, timed one-day travel itinerary based on the user's **city, budget, and interests**, while handling invalid inputs, API errors, and an unrealistically low budget.

2. Tools and Technologies Required

Hardware

- Computer/Laptop
- Internet connection

Software

- Visual Studio Code
- Python 3.x
- Python Virtual Environment (venv)
- Google Gemini API
- .env file for secure API key storage

Python Libraries

google-genai

python-dotenv

Install libraries

pip install google-genai python-dotenv

3. API Details

Parameter	Value
Platform	Google Gemini API
Model	gemini-3.6-flash
API Method	client.models.generate_content()
Temperature	0.3
Response Format	application/json
Max Tokens	Not explicitly specified
Authentication	GEMINI_API_KEY from .env

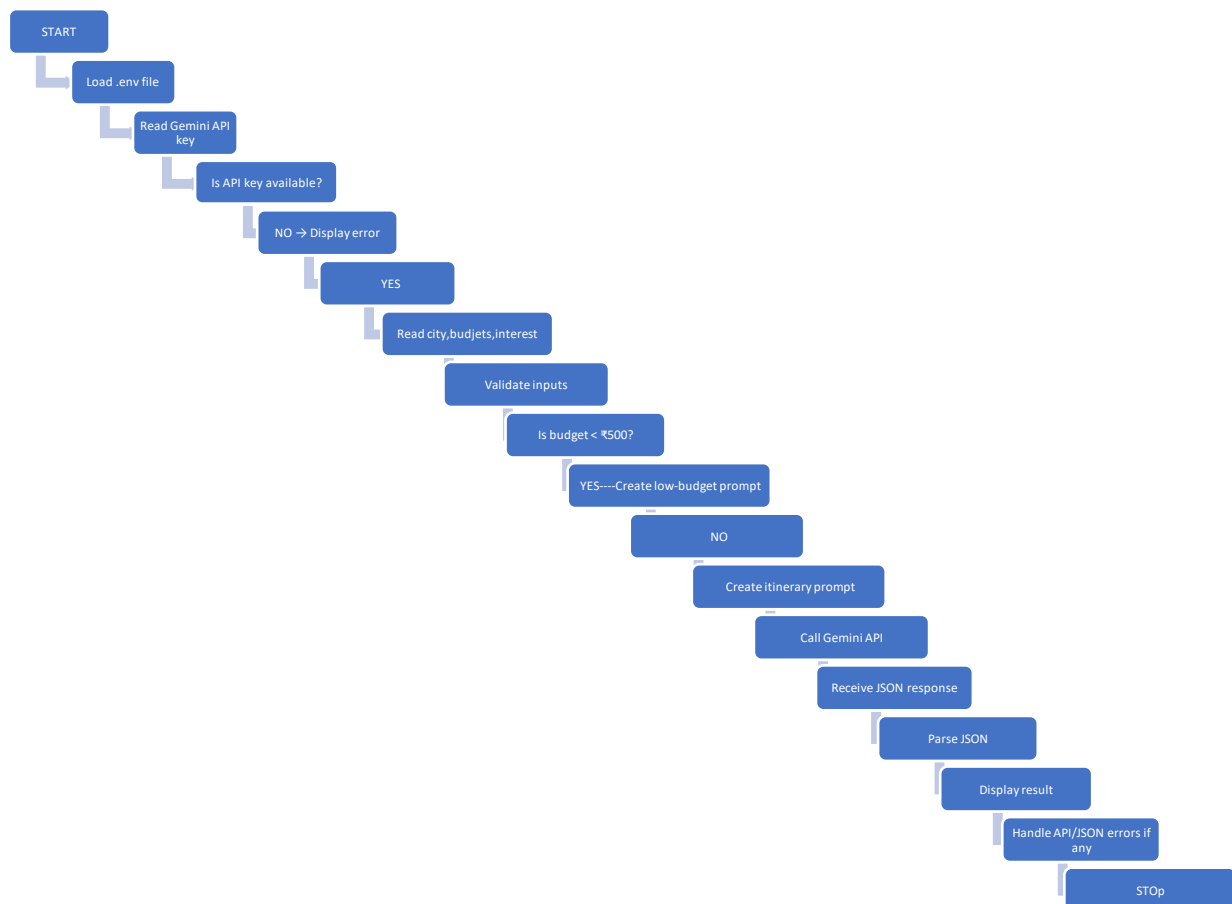
4. Procedure:

Create: travel_itinerary.py

The program:

1. Loads the .env file.
2. Reads the Gemini API key.
3. Accepts city, budget and interests.
4. Validates the inputs.
5. Creates a prompt.
6. Sends the prompt to Gemini.
7. Requests JSON output.
8. Parses the JSON using json.loads().
9. Displays the itinerary.
10. Handles errors.

5. Algorithm



Algorithm:

1. Start the program.
2. Load the Gemini API key from the .env file.
3. Read the city, budget, and interests from the user.
4. Validate that the city and interests are not empty.
5. Validate that the budget is a positive number.
6. Check whether the budget is unrealistically low.
7. Create a prompt containing the city, budget, interests, and itinerary requirements.
8. Send the prompt to the Gemini API using `client.models.generate_content()`.
9. Request the response in JSON format.
10. Parse the JSON response using `json.loads()`.
11. Display the city, budget, interests, budget status, and itinerary.
12. For an unrealistically low budget, display affordable individual options instead of a full-day itinerary.
13. Handle API failures, timeouts, quota/rate-limit errors, invalid JSON, and missing input.
14. Stop the program.

6. Working Principle

The application follows a **request** → **response** → **parsing** → **presentation** workflow.

Request

The user's details are inserted into a prompt:

City: Chennai

Budget: INR 3000

Interests: food

API Call

The Gemini API receives the prompt using:

```
client.models.generate_content()
```

Response

Gemini returns structured JSON.

Parsing

Python converts the JSON string into a Python dictionary using:

```
json.loads(response.text)
```

Presentation

The program extracts the fields and displays:

- City
- Budget
- Interests
- Budget status
- Time
- Activity
- Estimated cost
- Total estimated cost

7. Test Case 1 — Chennai

Input

Enter city: Chennai

Enter budget in INR: 3000

Enter interests separated by commas: food

```
ers\dnara\OneDrive\Desktop\CSA6583\CO3 AI1\travel_itinerary.py
Enter city: chennai
Enter budget in INR: 3000
Enter interests separated by commas (e.g., history, food, beach): food

TRAVEL ITINERARY
=====
City: chennai
Budget: INR 3000.0
Interests: food
Budget Status: sufficient
Realistic Minimum Budget: INR 1500.0

ITINERARY
-----

Time: 07:30 AM - 08:30 AM
Activity: Traditional South Indian breakfast at Murugan Idli Shop in T. Nagar, enjoying soft ghee podi idlis, medu vada, and authentic degree filter coffee.
Estimated Cost: INR 250

Time: 09:00 AM - 10:30 AM
Activity: Food heritage walk around Mylapore Kapaleeshwarar Temple streets, stopping for famous chilled rose milk at Kalathi Rose Milk Shop.
Estimated Cost: INR 100

Time: 11:00 AM - 12:00 PM
Activity: Mid-morning traditional snack break at the historic Rayar's Mess for hot rava dosa, bajji, and filter coffee.
Estimated Cost: INR 150

Time: 01:00 PM - 02:30 PM
Activity: Hearty Tamil non-veg meals lunch at Nair Mess or Chettinad feast at Junior Kuppanna, featuring authentic spicy curries and seafood.
Estimated Cost: INR 650

Time: 04:30 PM - 06:30 PM
Activity: Street food crawl in Sowcarpet tasting famous local fusion dishes like Murukku Sandwich, onion kachoris, and badam milk at Kakada Ramprasad.
Estimated Cost: INR 450

Time: 07:30 PM - 09:30 PM
Activity: Freshly fried coastal seafood dinner at Marina Beach stalls or a full coastal meal experience at Velu Military Hotel.
Estimated Cost: INR 900

-----
Total Estimated Cost: INR 2500
```

8. Test Case 2 — Delhi

Input

Enter city: Delhi

Enter budget in INR: 3000

Enter interests separated by commas: food

Output

```
Enter city: delhi
Enter budget in INR: 3000
Enter interests separated by commas (e.g., history, food, beach): food

TRAVEL ITINERARY
=====
City: delhi
Budget: INR 3000.0
Interests: food
Budget Status: sufficient
Realistic Minimum Budget: INR 1500.0

ITINERARY
-----

Time: 08:30 AM - 10:00 AM
Activity: Traditional North Indian breakfast of Bedmi Puri, Nagori Halwa, and Sweet Lassi at Shyam Sweets in Chandni Chowk.
Estimated Cost: INR 250

Time: 10:30 AM - 12:00 PM
Activity: Old Delhi food walk sampling hot Jalebis at Old Famous Jalebi Wala and Daulat Ki Chaat (seasonal dessert).
Estimated Cost: INR 250

Time: 01:00 PM - 02:30 PM
Activity: Authentic Mughlai lunch at Al Jawahar near Jama Masjid, trying Mutton Qorma, Seekh Kebabs, and Khameeri Roti.
Estimated Cost: INR 800

Time: 04:30 PM - 06:00 PM
Activity: Evening refreshment at Connaught Place: Classic milkshake at Keventers and street chaat at Prabhu Chaat Bhandar.
Estimated Cost: INR 350

Time: 07:00 PM - 08:30 PM
Activity: Street food tour at Lajpat Nagar Central Market sampling famous Ram Ladoo, Kurkure Momos, and Kathi Rolls.
Estimated Cost: INR 350

Time: 09:00 PM - 10:30 PM
Activity: Dinner at Havemore on Pandara Road, enjoying world-famous Butter Chicken, Dal Makhani, and Garlic Naan.
Estimated Cost: INR 900

-----
Total Estimated Cost: INR 2900.0
=====
```

9. Test Case 3 — Edge Case: Unrealistically Low Budget

Input

Enter city: Chennai

Enter budget in INR: 100

Enter interests separated by commas: food

Output

```
PS (dhara@OneDrive\Desktop\CS\6503) PS: AT1(travel_itinerary.py)
Enter city: chennai
Enter budget in INR: 100
Enter interests separated by commas (e.g., history, food, beach): food

WARNING: Budget is low for a complete one-day itinerary.
Affordable individual food options will be suggested.

TRAVEL ITINERARY
=====
City: chennai
Budget: INR 100.0
Interests: food
Budget Status: insufficient for full-day itinerary
Realistic Minimum Budget: INR 600

AFFORDABLE FOOD OPTIONS
-----
Choose ANY ONE option that fits your budget:

1. Filter Coffee and Hot Idlis
   Description: Enjoy traditional South Indian breakfast featuring soft steamed rice cakes served with sambar and chutneys, followed by authentic Madras filter coffee.
   Estimated Cost: INR 60

2. Burmese Atho at North Chennai Street Stalls
   Description: Try Atho, a popular Chennai street food adapted from Burmese cuisine, consisting of tossed orange noodles with cabbage, garlic oil, and crispy fried onions.
   Estimated Cost: INR 70

3. Beachside Sundal and Bhajji
   Description: Savor warm seasoned chickpea/corn sundal and freshly fried banana or chilli bhajjis at Marina Beach.
   Estimated Cost: INR 50

4. Ghee Podi Dosa at a Local Tiffin Center
   Description: Crispy fermented crepe smeared with aromatic spicy lentil powder (podi) and ghee, served with coconut chutney.
   Estimated Cost: INR 80

=====
(venv) PS: C:\Users\dhara\OneDrive\Desktop\CS\6503>
```

10. Model and Parameter Justification

Model

gemini-3.6-flash was selected because the application requires fast text generation and structured JSON output. The task does not require extensive reasoning, so a Flash model is appropriate.

Temperature

temperature = 0.3

A low temperature provides more consistent and predictable responses. This is useful because the application needs structured, budget-conscious itineraries rather than highly creative responses.

JSON Response

```
"response_mime_type": "application/json"
```

This ensures that the model returns JSON, which can be processed programmatically using:

```
json.loads()
```

Max Tokens

No explicit maximum-token value was specified in the program. Therefore, the model's default output-token limit is used.

11. Error Handling

The program handles different types of errors:

- **Missing API key:** Displays an error when GEMINI_API_KEY is not found in .env.
- **Invalid city:** Rejects an empty city.
- **Invalid budget:** Rejects non-numeric, zero, or negative budgets.
- **Missing interests:** Rejects empty interests.
- **API failure:** Displays an error when the Gemini API request fails.
- **Timeout:** Informs the user when the API request times out.
- **Rate limit/quota:** Displays an appropriate message when the API quota or rate limit is exceeded.
- **Invalid JSON:** Handles cases where the Gemini response cannot be parsed as JSON.
- **Missing fields:** Handles missing required fields in the Gemini response.

12. Result

The **Travel Itinerary Drafter** was successfully implemented using the **Gemini API**. The application accepts a city, budget, and interests and generates a **timed one-day itinerary in structured JSON**. It successfully demonstrated itinerary generation for **Chennai and Delhi** and correctly handled the **₹100 unrealistically low-budget edge case** by providing affordable individual food options instead of exceeding the available budget.

13. Conclusion

Thus, the Gemini API was successfully integrated into a Python-based travel itinerary application. Secure authentication, input validation, API communication, JSON parsing, structured output, budget handling, and error handling were implemented successfully. The system can generate personalized travel plans while adapting its response according to the user's available budget.